

**ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA TOÁN - ỨNG DỤNG**

—oOo—



**BÀI TẬP
HỌC SÂU**

ĐỀ TÀI: BÁO CÁO LAB01

MÔN HỌC: HỌC SÂU

MÃ HỌC PHẦN: 858048

SINH VIÊN THỰC HIỆN: TRƯƠNG TUYẾT NGHI

LỚP: KHOA HỌC DỮ LIỆU - DDU1231

GIẢNG VIÊN: ĐỖ NHƯ TÀI

TP. Hồ Chí Minh, tháng 09, năm 2026

MỤC LỤC

1	Tạo Tensor cơ bản	2
2	Tạo các Tensor thường dùng	2
3	Kiểm tra thông tin Tensor	3
4	Slicing và Indexing	4
5	Reshape và Flatten	5
6	Phép toán và nhân ma trận	6
7	Các hàm thống kê trên Tensor	8
8	Autograd cơ bản	9
9	Gradient Descent tìm hệ số đa thức	10
10	Giải hệ phương trình bằng Autograd	11
11	Bài tập tổng hợp	12

Bài 1: Tạo Tensor cơ bản

Tạo một Tensor số nguyên kích thước 2×3 và hiển thị kết quả.

Code:

```
1 import torch
2
3 a = torch.tensor(
4     [[1, 2, 3],
5     [4, 5, 6]],
6     dtype=torch.int32
7 )
8
9 print(a)
```

Output:

```
tensor([[1, 2, 3],
        [4, 5, 6]], dtype=torch.int32)
```

Yêu cầu: Cho biết kích thước, số chiều và kiểu dữ liệu của Tensor a.

Kết quả:

- Kích thước (Shape): 2×3 .
- Số chiều (ndim): 2 chiều (gồm 2 hàng và 3 cột).
- Kiểu dữ liệu (dtype): Số nguyên (int32).

Bài 2: Tạo các Tensor thường dùng

Thực hành tạo Tensor ngẫu nhiên, Tensor toàn số 0, toàn số 1, ma trận đơn vị

Code:

```
1 import torch
2
3 a = torch.rand(3, 4)
4 b = torch.randn(3, 4)
5 c = torch.zeros(2, 3)
6 d = torch.ones(2, 3)
7 e = torch.eye(4)
8
9 print(a)
10 print(b)
11 print(c)
12 print(d)
13 print(e)
```

Output:

```
tensor([[0.6564, 0.2357, 0.0230, 0.2447],
        [0.2288, 0.0760, 0.0233, 0.6651],
        [0.5633, 0.0643, 0.6013, 0.8193]])
tensor([[ 0.0746, -0.4413,  0.1134,  0.9300],
        [ 1.2199, -1.2352,  0.1010, -0.2774],
        [ 0.3536, -0.3021, -0.0517, -0.5805]])
```

```

tensor([[0., 0., 0.],
        [0., 0., 0.]])
tensor([[1., 1., 1.],
        [1., 1., 1.]])
tensor([[1., 0., 0., 0.],
        [0., 1., 0., 0.],
        [0., 0., 1., 0.],
        [0., 0., 0., 1.]])

```

Yêu cầu:

- Nhận xét sự khác nhau giữa `torch.rand()` và `torch.randn()`.
- Tạo thêm một Tensor 5×5 chứa toàn giá trị 10.

Kết quả:

- Sự khác nhau giữa `torch.rand()` và `torch.randn()`:
 - `torch.rand()`: Sử dụng phân phối đều $[0, 1)$, các giá trị trong ma trận chỉ nằm trong khoảng 0 đến 1, không chứa giá trị âm.
 - `torch.randn()`: Sử dụng phân phối chuẩn (Gauss) $(-\infty, +\infty)$, giá trị tập trung quanh 0, càng xa 0 càng ít xuất hiện và có chứa giá trị âm.
- Tensor 5×5 chứa toàn giá trị 10

```

1 # Tạo thêm một Tensor 5 x 5 chứa toàn giá trị 10.
2 f = torch.full((5, 5), 10)
3
4 print(f)

```

```

tensor([[10, 10, 10, 10, 10],
        [10, 10, 10, 10, 10],
        [10, 10, 10, 10, 10],
        [10, 10, 10, 10, 10],
        [10, 10, 10, 10, 10]])

```

Bài 3: Kiểm tra thông tin Tensor

Sử dụng `shape`, `size()`, `ndim` và `dtype` để kiểm tra các thuộc tính của Tensor.

Code:

```

1 import torch
2 a = torch.zeros(2, 3, 4)
3
4 print("Tensor:")
5 print(a)
6 print("Shape:", a.shape)
7 print("Size:", a.size())
8 print("Number of dimensions:", a.ndim)

```

Output:

```

Tensor:
tensor([[[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]],

```

```

[[0., 0., 0., 0.],
 [0., 0., 0., 0.],
 [0., 0., 0., 0.]])
Shape: torch.Size([2, 3, 4])
Size: torch.Size([2, 3, 4])
Number of dimensions: 3

```

Yêu cầu: Giải thích ý nghĩa của `torch.Size([2, 3, 4])`.

Kết quả:

- Tensor có kích thước $2 \times 3 \times 4$, gồm 3 chiều và tổng cộng $2 \times 3 \times 4 = 24$ phần tử.
- Tensor gồm 2 ma trận, mỗi ma trận có 3 hàng và 4 cột. Các phần tử đều bằng 0.

Bài 4: Slicing và Indexing

Truy xuất một phần dữ liệu từ Tensor nhiều chiều.

Code:

```

1 import torch
2 a = torch.randn(3, 4, 5)
3 print("Original tensor:")
4 print(a)
5 print("\nTensor a[1]:")
6 print(a[1])
7 print("\nTensor a[1:, 2:4]:")
8 print(a[1:, 2:4])

```

Output:

```

Original tensor:
tensor([[[ 0.1507, -1.8328, -1.6220, -0.2177,  0.9911],
          [ 1.0336,  0.3107,  0.1657,  0.9995, -0.2127],
          [ 0.6844,  0.9617,  1.3829,  0.7071, -0.0508],
          [-0.1017, -0.2587,  1.9031, -0.4535, -0.3880]],

        [[-0.9257,  0.2646, -0.7212, -0.5850,  0.8321],
          [-0.4802, -0.3190,  0.6407,  0.9775, -0.2529],
          [-0.1893, -0.3397, -0.0297, -1.0659, -0.9922],
          [ 0.6998, -0.1418, -0.3899, -0.1365,  3.0168]],

        [[ 0.3761, -0.9744,  0.5913,  1.0597, -0.0846],
          [ 1.3485,  2.4708,  0.6384, -0.6667,  1.5369],
          [ 0.4540, -0.0232,  0.3527,  0.1497, -0.3582],
          [-0.1223,  0.5231, -0.3172,  0.7384, -0.4826]]])

Tensor a[1]:
tensor([[-0.9257,  0.2646, -0.7212, -0.5850,  0.8321],
        [-0.4802, -0.3190,  0.6407,  0.9775, -0.2529],
        [-0.1893, -0.3397, -0.0297, -1.0659, -0.9922],
        [ 0.6998, -0.1418, -0.3899, -0.1365,  3.0168]])

```

```
Tensor a[1:, 2:4]:
tensor([[[[-0.1893, -0.3397, -0.0297, -1.0659, -0.9922],
          [ 0.6998, -0.1418, -0.3899, -0.1365,  3.0168]],

        [[ 0.4540, -0.0232,  0.3527,  0.1497, -0.3582],
          [-0.1223,  0.5231, -0.3172,  0.7384, -0.4826]]]])
```

Yêu cầu:

- Cho biết shape của `a[1]` và `a[1:, 2:4]`.
- Thử lấy hai phần tử cuối cùng trên chiều thứ ba.

Kết quả:

- Với Tensor `a` có shape `(3,4,5)`, `a[1]` lấy ma trận thứ hai nên có shape `(4,5)`.

```
1 #Shape
2 print ("Shape a[1]: " , (a[1].shape))
3
4 print ("Shape a[1:, 2:4]: " , (a[1:, 2:4].shape))
```

```
Shape a[1]: torch.Size([4])
Shape a[1:, 2:4]: torch.Size([2, 2])
```

- `a[1:, 2:4]` lấy hai ma trận cuối, giữ từ hàng 3 5 cột, nên có shape `(2,2,5)`.

```
1 # Lấy hai phần tử cuối cùng trên chiều thứ ba
2 a[:, :, -2:]
```

```
tensor([[[[-0.2177,  0.9911],
          [ 0.9995, -0.2127],
          [ 0.7071, -0.0508],
          [-0.4535, -0.3880]],

        [[-0.5850,  0.8321],
          [ 0.9775, -0.2529],
          [-1.0659, -0.9922],
          [-0.1365,  3.0168]],

        [[ 1.0597, -0.0846],
          [-0.6667,  1.5369],
          [ 0.1497, -0.3582],
          [ 0.7384, -0.4826]]]])
```

Bài 5: Reshape và Flatten

Thay đổi hình dạng Tensor mà không làm thay đổi số phần tử.

Code:

```
1 import torch
2 a = torch.randn(3, 4)
3 print("Original:")
4 print(a)
5 print("\nFlatten:")
6 print(a.ravel())
7 print("\nReshape to 3 x 2 x 2:")
```

```
8 print(a.reshape(3, 2, 2))
```

Output:

Original:

```
tensor([[ 0.0294, -1.1352,  1.4799,  1.6398],
        [ 0.1498,  1.9367,  0.8845,  1.4105],
        [ 0.7305,  1.1645,  0.8977,  0.4912]])
```

Flatten:

```
tensor([ 0.0294, -1.1352,  1.4799,  1.6398,  0.1498,  1.9367,  0.8845,
        ↪  1.4105,
        0.7305,  1.1645,  0.8977,  0.4912])
```

Reshape to 3 x 2 x 2:

```
tensor([[[ 0.0294, -1.1352],
         [ 1.4799,  1.6398]],

        [[ 0.1498,  1.9367],
         [ 0.8845,  1.4105]],

        [[ 0.7305,  1.1645],
         [ 0.8977,  0.4912]]])
```

Yêu cầu:

- Kiểm tra số phần tử trước và sau khi reshape.
- Thử reshape Tensor thành kích thước 2×6 .

Kết quả:

- Số phần tử trước và sau khi reshape.
 - Số phần tử trước reshape: 12 phần tử,
 - Số phần tử sau reshape: 3 ma trận đều có 4 phần tử.

```
1 # số phần tử trước và sau khi reshape.
2 print("Trước reshape:", a.shape)
3 print("Sau reshape:", a.reshape(3, 2, 2).shape)
```

```
Trước reshape: torch.Size([3, 4])
Sau reshape: torch.Size([3, 2, 2])
```

- Reshape Tensor thành kích thước 2×6 .

```
1 # reshape Tensor thành kích thước 2 x 6
2 print(a.reshape( 2, 6))

tensor([[ 0.0294, -1.1352,  1.4799,  1.6398,  0.1498,  1.9367],
        [ 0.8845,  1.4105,  0.7305,  1.1645,  0.8977,  0.4912]])
```

Bài 6: Phép toán và nhân ma trận

Thực hiện các phép toán phần tử và phép nhân ma trận bằng toán tử @.

Code:

```

1 import torch
2
3 a = torch.randn(2, 3)
4 b = torch.randn(2, 3)
5
6 print("a:")
7 print(a)
8 print("b:")
9 print(b)
10
11 print("\na + b:")
12 print(a + b)
13 print("\na / b:")
14 print(a / b)
15
16 print("\na^2:")
17 print(a ** 2)
18
19 print("\nMatrix multiplication:")
20 print(a @ b.T)

```

Output:

```

a:
tensor([[ 0.4173,  0.7319, -1.0494],
        [ 2.0330, -1.5014,  1.7079]])
b:
tensor([[ 0.4472,  0.6720,  0.9830],
        [-1.1775, -0.2348, -1.0270]])

a + b:
tensor([[ 0.8646,  1.4039, -0.0663],
        [ 0.8556, -1.7361,  0.6809]])

a / b:
tensor([[ 0.9331,  1.0891, -1.0674],
        [-1.7266,  6.3952, -1.6630]])

a^2:
tensor([[0.1742, 0.5357, 1.1011],
        [4.1332, 2.2541, 2.9168]])

Matrix multiplication:
tensor([[ -0.3531,  0.4145],
        [ 1.5793, -3.7953]])

```

Yêu cầu:

- Phân biệt phép toán $a * b$ với $a @ b.T$.
- Cho biết shape của kết quả $a @ b.T$.

Kết quả:

- $a * b$: nhân từng cặp phần tử ở cùng vị trí. Hai Tensor có shape (2,3) nên kết quả cũng có shape (2,3).
- $a @ b.T$: là phép nhân ma trận. Tensor $b.T$ có shape (3,2) nên tích của ma trận (2,3) với

ma trận (3,2) có shape (2,2).

```
1 # Ma trận a*b
2 print(a*b)
3
4 # Ma trận a @ b.T
5 print(a @ b.T)
6
7 #Shape a @ b.T
8 print("Shape a @ b.T: ", (a @ b.T).shape)

tensor([[ 0.1867,  0.4918, -1.0316],
        [-2.3938,  0.3525, -1.7540]])
tensor([[ -0.3531,  0.4145],
        [ 1.5793, -3.7953]])
Shape a @ b.T:  torch.Size([2, 2])
```

Bài 7: Các hàm thống kê trên Tensor

Tính trung bình, độ lệch chuẩn và tổng tích lũy theo từng cột.

Code:

```
1 import torch
2 a = torch.randn(3, 4)
3 print(a)
4 print("\nMean:")
5 print(torch.mean(a, dim=0))
6 print("\nStandard deviation:")
7 print(torch.std(a, dim=0))
8 print("\nCumulative sum:")
9 print(torch.cumsum(a, dim=0))
```

Output:

```
tensor([[ 0.2428, -2.0160, -1.1070, -0.3504],
        [-0.3763,  0.1236, -0.5444, -0.7849],
        [-0.4414, -1.3558,  0.2760,  1.2377]])
```

Mean:

```
tensor([-0.1916, -1.0827, -0.4585,  0.0341])
```

Standard deviation:

```
tensor([0.3777, 1.0956, 0.6955, 1.0647])
```

Cumulative sum:

```
tensor([[ 0.2428, -2.0160, -1.1070, -0.3504],
        [-0.1335, -1.8924, -1.6515, -1.1353],
        [-0.5749, -3.2482, -1.3755,  0.1023]])
```

Yêu cầu: Thay dim=0 bằng dim=1 và nhận xét sự khác nhau.

Kết quả:

- dim=0 tính theo từng cột, nên mean và std trả về 4 giá trị; cumsum cộng dồn từ trên xuống.
- dim=1 tính theo từng hàng, nên mean và std trả về 3 giá trị; cumsum cộng dồn từ trái sang phải.

- Trong cả hai trường hợp, `cumsum` giữ nguyên shape (3,4) của Tensor đầu vào.

```

1 # dim = 1
2 print(torch.mean(a, dim=1))
3
4 print("\nStandard deviation:")
5 print(torch.std(a, dim=1))
6
7 print("\nCumulative sum:")
8 print(torch.cumsum(a, dim=1))

```

tensor([-0.8077, -0.3955, -0.0709])

Standard deviation:
tensor([0.9768, 0.3846, 1.0986])

Cumulative sum:
tensor([[0.2428, -1.7732, -2.8803, -3.2307],
 [-0.3763, -0.2527, -0.7972, -1.5821],
 [-0.4414, -1.7972, -1.5212, -0.2836]])

Bài 8: Autograd cơ bản

Tạo biến có `requires_grad=True` và dùng `backward()` để tính đạo hàm tự động.

Code:

```

1 import torch
2 x = torch.tensor(
3     3.6,
4     requires_grad=True
5 )
6 y = x * x
7 y.backward()
8 print("x =", x)
9 print("y =", y)
10 print("x.grad =", x.grad)

```

Output:

```

x = tensor(3.6000, requires_grad=True)
y = tensor(12.9600, grad_fn=<MulBackward0>)
x.grad = tensor(7.2000)

```

Yêu cầu:

- Tính thủ công đạo hàm của $y = x^2$ tại $x = 3.6$ và so sánh với `x.grad`.
- Đổi hàm thành $y = 3x^2 + 2x + 1$ và kiểm tra gradient.

Kết quả:

- Với $y = x^2$, ta có $y' = 2x$. Tại $x = 3.6$, đạo hàm bằng 7.2, khớp với giá trị `tensor(7.2000)` được in ra.
- Với $y = 3x^2 + 2x + 1$, ta có $y' = 6x + 2$. Tại $x = 3.6$, đạo hàm bằng 23.6. Với `x.grad() = 23.6000`, kết quả bằng nhau.

```

1 # y=3x^2+2x+1, x= 3.6

```

```

2 x = torch.tensor(
3 3.6,
4 requires_grad=True
5 )
6 y = 3*x**2 + 2*x + 1
7 y.backward()
8 print("x =", x)
9 print("y =", y)
10 print("x.grad =", x.grad)

x = tensor(3.6000, requires_grad=True)
y = tensor(47.0800, grad_fn=<AddBackward0>)
x.grad = tensor(23.6000)

```

Bài 9: Gradient Descent tìm hệ số đa thức

Sinh dữ liệu từ đa thức $y = x^2 + 2x + 3$, sau đó dùng NAdam và autograd để tìm lại ba hệ số.

Code:

```

1 import numpy as np
2 import torch
3
4 polynomial = np.poly1d([1, 2, 3])
5 N = 20
6
7 X = np.random.randn(N, 1) * 5
8 Y = polynomial(X)
9
10 XX = np.hstack([
11 X * X,
12 X,
13 np.ones_like(X)
14 ])
15
16 w = torch.randn(3, 1, requires_grad=True)
17 x = torch.tensor(XX, dtype=torch.float32)
18 y = torch.tensor(Y, dtype=torch.float32)
19
20 optimizer = torch.optim.NAdam([w], lr=0.01)
21
22 print("Initial coefficients:")
23 print(w)
24
25 for epoch in range(1000):
26     optimizer.zero_grad()
27     y_pred = x @ w
28     mse = torch.mean(torch.square(y - y_pred))
29     mse.backward()
30     optimizer.step()
31
32 print("Learned coefficients:")
33 print(w)

```

Output:

```

Initial coefficients:
tensor([[ -1.8516],
        [ 0.9187],

```

```

        [-0.0345]], requires_grad=True)
Learned coefficients:
tensor([[1.0014],
        [2.0022],
        [2.8323]], requires_grad=True)

```

Yêu cầu:

- So sánh ba hệ số học được với $[1, 2, 3]$.
- Giải thích `optimizer.zero_grad()`, `mse.backward()` và `optimizer.step()`.

Kết quả:

- So sánh ba hệ số học được với $[1, 2, 3]$.
 - Theo output, ba hệ số học được là $[1.0014, 2.0022, 2.8323]$, tương ứng với các hệ số của x^2 , x và hệ số tự do.
 - So với $[1, 2, 3]$, sai lệch xấp xỉ 0.0014, 0.0022 và 0.1677. Hai hệ số đầu đã gần giá trị ban đầu hơn hệ số tự do; sau 1000 epoch, các hệ số chưa trùng hoàn toàn với đa thức ban đầu.
- Giải thích `optimizer.zero_grad()`, `mse.backward()` và `optimizer.step()`.
 - `optimizer.zero_grad()`: Reset toàn bộ đạo hàm của các biến hiện tại về ban đầu trước khi tính đạo hàm mới.
 - `mse.backward()`: Tính đạo hàm của hàm mất mát (MSE) theo các hệ số w .
 - `optimizer.step()`: cập nhật hệ số w theo thuật toán NAdam để giảm sai số MSE.

Bài 10: Giải hệ phương trình bằng Autograd

Tối thiểu hóa tổng sai số bình phương để tìm A, B, C, D thỏa mãn bốn phương trình

Code:

```

1 import random
2 import torch
3
4 A = torch.tensor(random.random(), requires_grad=True)
5 B = torch.tensor(random.random(), requires_grad=True)
6 C = torch.tensor(random.random(), requires_grad=True)
7 D = torch.tensor(random.random(), requires_grad=True)
8
9 EPOCHS = 2000
10 optimizer = torch.optim.NAdam([A, B, C, D], lr=0.01)
11
12 for _ in range(EPOCHS):
13     y1 = A + B - 9
14     y2 = C - D - 1
15     y3 = A + C - 8
16     y4 = B - D - 2
17
18     sqerr = y1*y1 + y2*y2 + y3*y3 + y4*y4
19
20     optimizer.zero_grad()
21     sqerr.backward()
22     optimizer.step()

```

```

23
24 print("A =", A)
25 print("B =", B)
26
27 print("C =", C)
28 print("D =", D)

```

Output:

```

A = tensor(4.6465, requires_grad=True)
B = tensor(4.3534, requires_grad=True)
C = tensor(3.3535, requires_grad=True)
D = tensor(2.3534, requires_grad=True)

```

Yêu cầu:

- Kiểm tra nghiệm thu được với bốn phương trình đã cho.
- Nhận xét vì sao bài toán có thể có nhiều nghiệm.

Kết quả:

- Kiểm tra nghiệm thu được với bốn phương trình đã cho:

$$A + B = 8.9999, C - D = 1.0001, A + C = 8.0000, B - D = 2.0000.$$

Sai số rất nhỏ.

- Bài toán có thể có nhiều nghiệm:

Lấy phương trình thứ nhất cộng phương trình thứ hai rồi trừ phương trình thứ ba thu được $B - D = 2$, chính là phương trình thứ tư. Do đó hệ chỉ có 3 phương trình độc lập với 4 ẩn.

Đặt $D = t$, nghiệm tổng quát là $A = 7 - t$, $B = t + 2$, $C = t + 1$, $D = t$ với t tùy ý thuộc tập số thực. Hệ có vô số nghiệm nên kết quả tối ưu là một nghiệm gần đúng trong họ nghiệm này.

Bài 11: Bài tập tổng hợp

Yêu cầu:

- Tạo Tensor X có 24 phần tử và reshape lần lượt thành 4×6 , 2×12 và $2 \times 3 \times 4$.
- Tạo hai Tensor A, B kích thước 3×4 ; tính $A+B$, $A*B$, A/B , trung bình theo cột và độ lệch chuẩn theo cột.
- Cho $y = 2x^2 + 5x + 3$. Dùng autograd tính dy/dx tại $x = 2$ và kiểm tra bằng phép tính thủ công.
- Thay đa thức trong Bài 9 bằng $y = 2x^2 - 3x + 5$; huấn luyện và cho biết các hệ số tìm được.

Kết quả:

1. Tạo Tensor X có 24 phần tử và reshape lần lượt thành 4×6 , 2×12 và $2 \times 3 \times 4$.

```

1 # Tạo Tensor X có 24 phần tử và reshape lần lượt thành 4 x 6, 2 x 12 và 2 x 3 x
  → 4.
2 X = torch.randn(24)

```

```

3 print("Tensor X:")
4 print(X)
5
6 print("Tensor 4x6")
7 print(X.reshape(4, 6))
8
9 print("Tensor 2x12")
10 print(X.reshape(2, 12))
11
12 print("Tensor 2x3x4")
13 print(X.reshape(2, 3, 4))

```

```

Tensor X:
tensor([ 0.4969,  0.1460, -0.7639, -0.8069,  0.5555, -0.9160,
        ↪ -0.3864, -0.9593,
        -1.1049,  0.8254,  0.3225, -0.6399,  0.5124,  0.7560,
        ↪ -0.3735, -0.6824,
        0.4968,  1.2440, -0.1487, -0.6026,  0.9703,  0.3479,
        ↪ 2.0880,  1.0546])

Tensor 4x6
tensor([[ 0.4969,  0.1460, -0.7639, -0.8069,  0.5555, -0.9160],
        [-0.3864, -0.9593, -1.1049,  0.8254,  0.3225, -0.6399],
        [ 0.5124,  0.7560, -0.3735, -0.6824,  0.4968,  1.2440],
        [-0.1487, -0.6026,  0.9703,  0.3479,  2.0880,  1.0546]])

Tensor 2x12
tensor([[ 0.4969,  0.1460, -0.7639, -0.8069,  0.5555, -0.9160,
        ↪ -0.3864, -0.9593,
        -1.1049,  0.8254,  0.3225, -0.6399],
        [ 0.5124,  0.7560, -0.3735, -0.6824,  0.4968,  1.2440,
        ↪ -0.1487, -0.6026,
        0.9703,  0.3479,  2.0880,  1.0546]])

Tensor 2x3x4
tensor([[[ 0.4969,  0.1460, -0.7639, -0.8069],
        [ 0.5555, -0.9160, -0.3864, -0.9593],
        [-1.1049,  0.8254,  0.3225, -0.6399]],

        [[ 0.5124,  0.7560, -0.3735, -0.6824],
        [ 0.4968,  1.2440, -0.1487, -0.6026],
        [ 0.9703,  0.3479,  2.0880,  1.0546]]])

```

Cả ba shape đều bảo toàn số phần tử: $4 \times 6 = 2 \times 12 = 2 \times 3 \times 4 = 24$.

2. Tạo hai Tensor A, B kích thước 3×4 ; tính $A+B$, $A*B$, A/B , trung bình theo cột và độ lệch chuẩn theo cột.

```

1 # Tạo hai Tensor A, B kích thước  $3 \times 4$ ; tính  $A+B$ ,  $A*B$ ,  $A/B$ , trung bình theo cột
  ↪ và độ lệch chuẩn theo cột.
2 A = torch.randn(3, 4)
3 B = torch.randn(3, 4)
4
5 print("A =", A)
6 print("B =", B)
7
8 print("\nA + B:", A + B)
9 print("A * B:", A * B)
10 print("A / B:", A / B)

```

```

11
12 print("Trung bình theo cột của A:", A.mean(dim=0))
13 print("Độ lệch chuẩn theo cột của A:", A.std(dim=0))

A = tensor([[ 0.0352, -0.4872,  1.7132, -0.4583],
            [ 1.0278, -0.1278,  0.5428, -0.9981],
            [-0.0067,  1.0875, -0.2630,  0.1779]])
B = tensor([[ 0.5839,  0.4976,  1.9256, -0.5047],
            [ 0.4847,  0.5146,  1.9256, -0.2380],
            [-0.0819, -0.3730, -2.2113,  1.4797]])

A + B: tensor([[ 0.6191,  0.0104,  3.6388, -0.9630],
            [ 1.5125,  0.3868,  2.4683, -1.2361],
            [-0.0886,  0.7145, -2.4742,  1.6576]])
A * B: tensor([[ 2.0560e-02, -2.4244e-01,  3.2990e+00,  2.3129e-01],
            [ 4.9815e-01, -6.5752e-02,  1.0451e+00,  2.3752e-01],
            [ 5.4959e-04, -4.0567e-01,  5.8150e-01,  2.6322e-01]])
A / B: tensor([[ 0.0603, -0.9790,  0.8897,  0.9082],
            [ 2.1207, -0.2483,  0.2819,  4.1946],
            [ 0.0820, -2.9156,  0.1189,  0.1202]])
Trung bình theo cột của A: tensor([ 0.3521,  0.1575,  0.6643,
    ↪ -0.4262])
Độ lệch chuẩn theo cột của A: tensor([0.5856, 0.8252, 0.9937,
    ↪ 0.5887])

```

3. Cho $y = 2x^2 + 5x + 3$. Dùng autograd tính dy/dx tại $x = 2$ và kiểm tra bằng phép tính thủ công.

```

1 #Cho y = 2x^2 + 5x + 3. Dùng autograd tính dy/dx tại x = 2 và kiểm tra bằng phép
  ↪ tính thủ công.
2 x = torch.tensor(2.0, requires_grad=True)
3 y = 2 * x**2 + 5*x + 3
4 y.backward()
5 print("x=",x)
6 print("y=",y)
7 print("x.grad:", x.grad)

x= tensor(2., requires_grad=True)
y= tensor(21., grad_fn=<AddBackward0>)
x.grad: tensor(13.)

```

Với $y = 2x^2 + 5x + 3$, ta có $dy/dx = 4x + 5$. Tại $x = 2$, $y = 21$ và $dy/dx = 13$, khớp với output `x.grad: tensor(13.)`.

4. Huấn luyện đa thức $y = 2x^2 - 3x + 5$

```

1 # Thay đa thức trong Bài 9 bằng y = 2x^2 - 3x + 5; huấn luyện và cho biết các hệ
  ↪ số tìm được.
2 import numpy as np
3 import torch
4
5 polynomial = np.poly1d([2, -3, 5])
6 N = 20
7
8 X = np.random.randn(N, 1) * 5
9 Y = polynomial(X)
10

```

```

11 XX = np.hstack([
12 X * X,
13 X,
14 np.ones_like(X)
15 ])
16
17 w = torch.randn(3, 1, requires_grad=True)
18 x = torch.tensor(XX, dtype=torch.float32)
19 y = torch.tensor(Y, dtype=torch.float32)
20
21 optimizer = torch.optim.NAdam([w], lr=0.01)
22
23 print("Initial coefficients:")
24 print(w)
25
26 for epoch in range(1000):
27     optimizer.zero_grad()
28     y_pred = x @ w
29     mse = torch.mean(torch.square(y - y_pred))
30     mse.backward()
31     optimizer.step()
32
33 print("Learned coefficients:")
34 print(w)

```

```

Initial coefficients:
tensor([[ -0.2890],
        [ -0.0745],
        [ 1.3110]], requires_grad=True)
Learned coefficients:
tensor([[ 2.0145],
        [-3.0065],
        [ 3.9068]], requires_grad=True)

```

Nhận xét:

- Output là các hệ số $[2.0145, -3.0065, 3.9068]$. So với giá trị đích $[2, -3, 5]$, sai lệch lần lượt xấp xỉ 0.0145, 0.0065 và 1.0932.
- Hai hệ số của x^2 và x đã khá gần giá trị đích, nhưng hệ số tự do còn lệch đáng kể so với 5. Có thể quá trình tối ưu chưa hội tụ sau 1000 epoch; sự khác biệt về độ lớn giữa các cột X^2 , X và cột toàn 1 cũng có thể ảnh hưởng đến tốc độ học từng hệ số.